

PC Performance Copilot

Local telemetry platform that diagnoses PC performance issues — AI explains what the data already proved.

A system-level performance monitoring and diagnosis tool that reads hardware and software metrics directly from the operating system, stores them over time, detects anomalies with rules and statistics, and uses an LLM agent only as the final explanation layer — not as the source of truth.

When your machine lags, you ask a plain-English question like *"Why does my PC slow down when opening VS Code?"* The app correlates your question with collected telemetry, ranks root causes by evidence, and returns an actionable diagnosis.

Table of Contents

- [Why This Project](#)
 - [Core Philosophy](#)
 - [How It Works](#)
 - [Architecture](#)
 - [Data Collection](#)
 - [Storage & Data Model](#)
 - [Analysis Engine](#)
 - [AI Agent Layer](#)
 - [User Experience](#)
 - [Tech Stack](#)
 - [Project Structure](#)
 - [Development Plan](#)
 - [MVP Success Criteria](#)
 - [Demo Scenarios](#)
 - [Privacy & Security](#)
 - [Future Roadmap](#)
 - [Skills Demonstrated](#)
 - [Getting Started](#)
 - [License](#)
-

Why This Project

Most "AI assistant" projects are thin wrappers around an LLM. This project inverts that model:

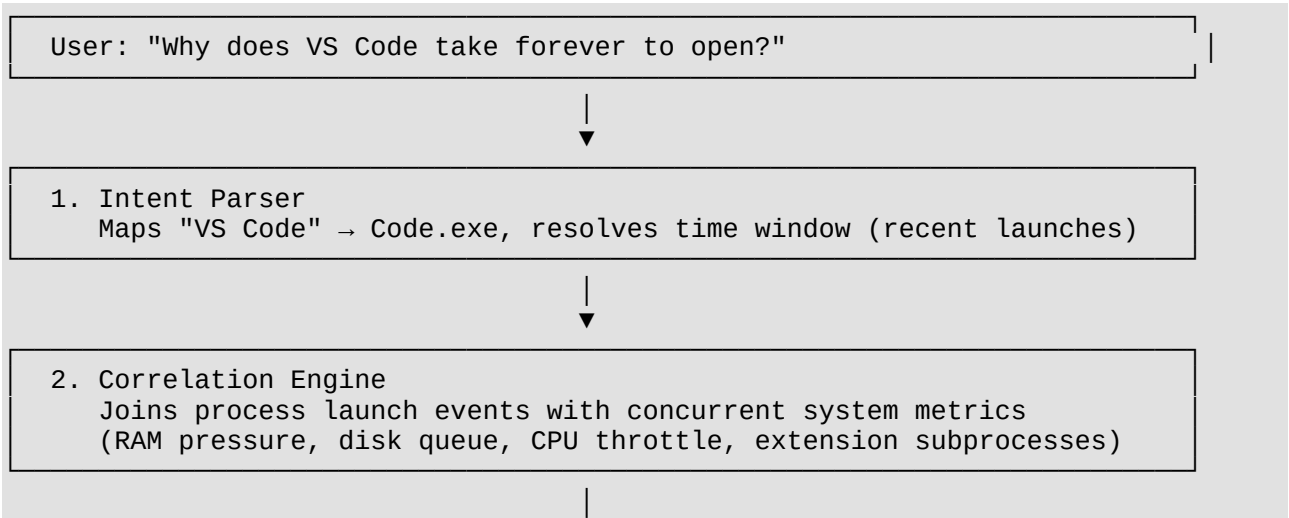
Typical AI project	PC Performance Copilot
LLM guesses the answer	Collectors gather ground truth from the OS
No persistent data	Time-series telemetry stored locally
Unverifiable output	Diagnosis backed by metrics and timestamps
Generic advice	Context-specific causes ranked by evidence

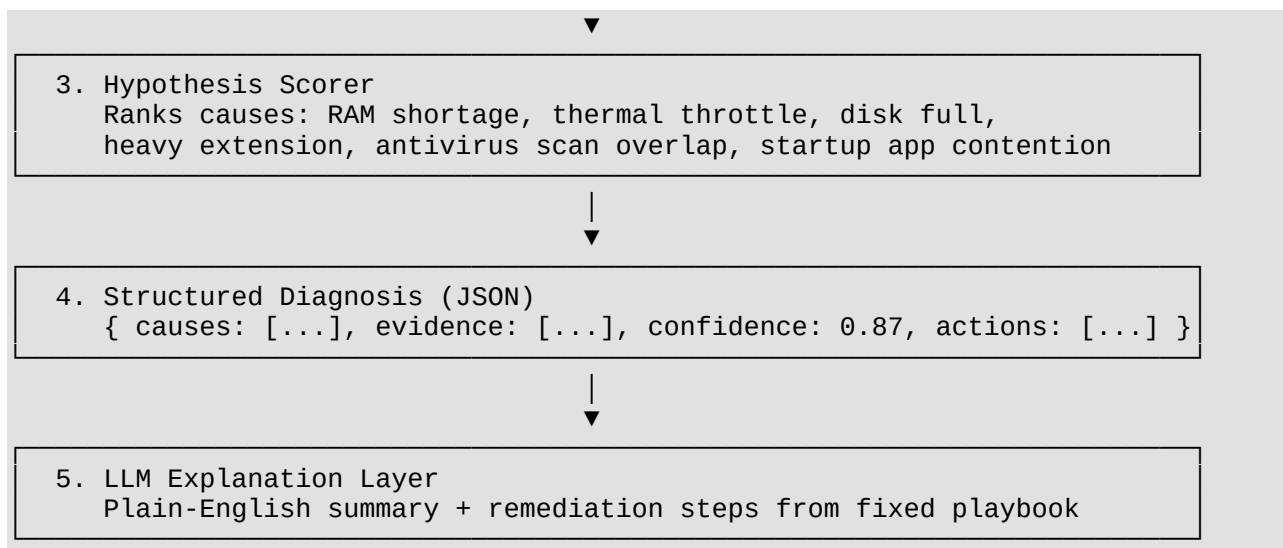
This maps to real-world engineering across **operating systems, performance monitoring, data engineering, anomaly detection, search/recommendation**, and **AI agents** — in one cohesive application that solves an everyday problem.

Core Philosophy

- Telemetry first, AI second** — The LLM never invents root causes. It receives a structured diagnosis payload and translates it into plain English.
- Local-first** — All data stays on the user's machine by default. No cloud upload required for core functionality.
- Evidence over intuition** — Every claim in a diagnosis links back to a metric, event, or log entry.
- Progressive depth** — Ship a narrow, demo-able vertical slice before expanding to benchmarking, cohort comparison, and game FPS analysis.
- Graceful degradation** — Missing sensors, denied permissions, or unsupported hardware reduce confidence scores rather than breaking the app.

How It Works





Example output:

VS Code startup took **8.2 seconds** (your 7-day median: **2.1s**, +290%).

During launch:

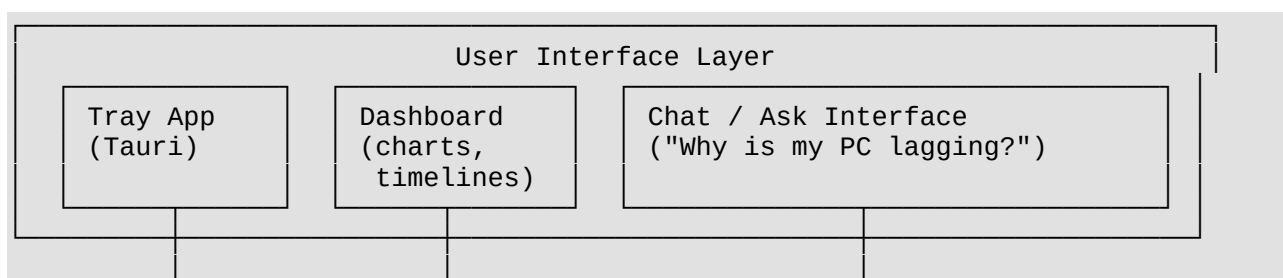
- Available RAM dropped to **412 MB** (94% memory pressure)
- Code . exe extension host (node . exe) peaked at **1.8 GB**
- Disk queue depth spiked to **14** (C: drive 96% full)

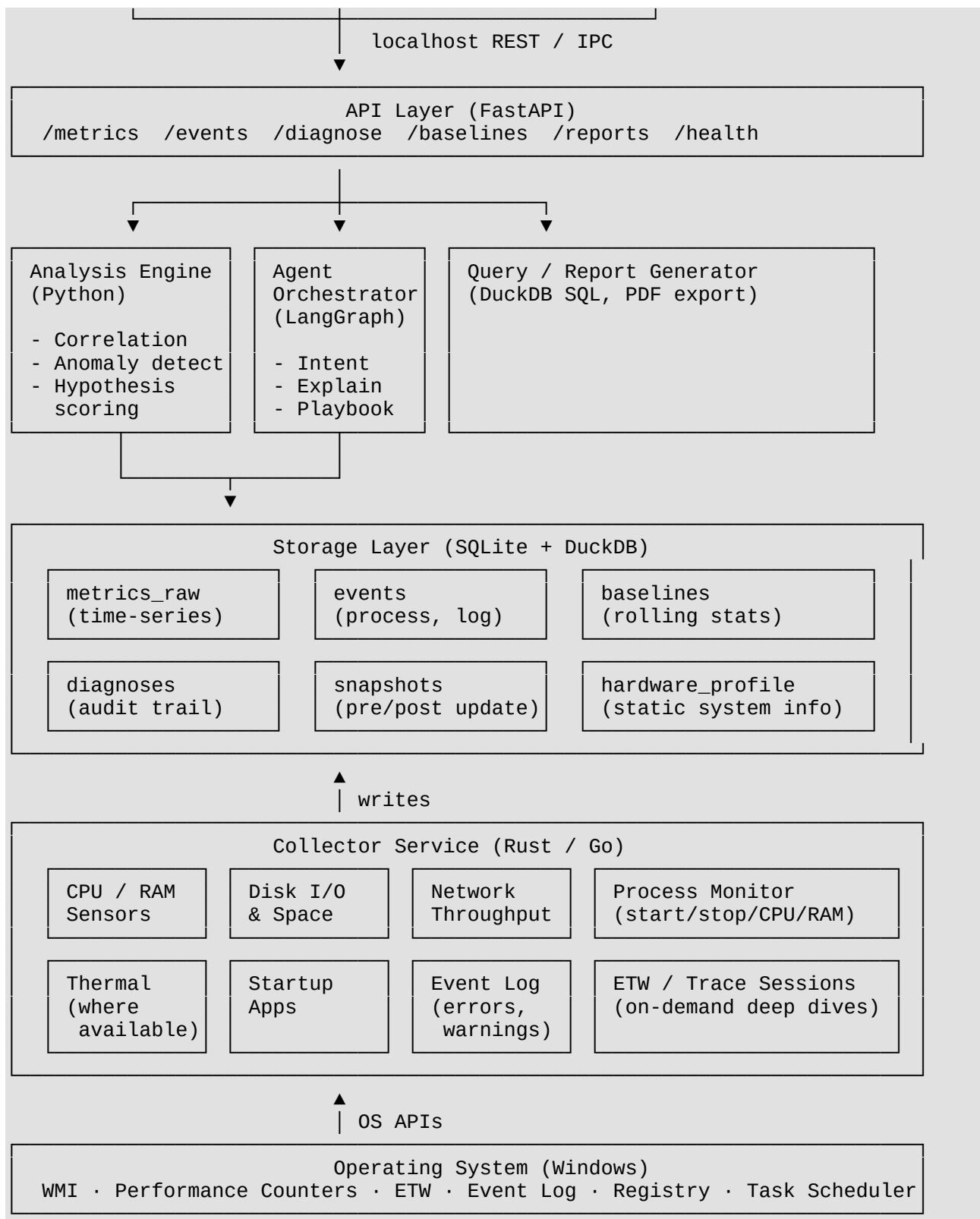
Most likely cause: Extension host memory pressure combined with low free disk space.

Suggested actions:

1. Disable or update the Python extension (top memory consumer in last 3 launches)
2. Free at least 10 GB on C: — less than 5% free severely impacts swap and temp files
3. Exclude %APPDATA%\Code from real-time antivirus scanning if safe for your environment

Architecture





Data Collection

Continuous metrics (polled every 1–5 seconds)

Metric	Source (Windows)	Notes
CPU utilization (total + per-core)	Performance Counters (\Processor(*))	Track sustained vs burst
Memory (total, available, committed)	\Memory\ counters	Detect memory pressure
Swap / page file usage	\Paging File(*)	Indicator of RAM exhaustion
Disk I/O (read/write bytes, queue depth)	\PhysicalDisk(*)	Queue depth > 2 = bottleneck signal
Disk free space (per volume)	WMI Win32_LogicalDisk	Alert below 10% / 5% thresholds
Network (bytes in/out per interface)	\Network Interface(*)	Detect background downloads
GPU utilization (optional)	DXGI / vendor APIs	Phase 2+
CPU temperature / throttle flags	WMI / ACPI (hardware-dependent)	Graceful fallback if unavailable

Event-driven collection

Event	Source	Trigger
Process start / stop	ETW Process provider or polling diff	Always
Process CPU/RAM snapshot	Per-process counters on start + every 5s for top N	Top 20 by resource usage
Application focus change	Win32 foreground window hook	Optional, Phase 2
Windows Event Log errors	System, Application logs	Errors and warnings
Service state changes	SCM notifications	Phase 2
Windows Update completed	Event Log ID 19 (WindowsUpdateClient)	Snapshot trigger
Driver install / update	SetupAPI event log	Snapshot trigger

On-demand deep collection

When a user asks about a specific app or a spike is detected:

- **ETW trace session** (30–60 seconds): disk I/O per process, CPU sampling, file I/O
- **Process tree snapshot**: parent/child relationships (e.g., VS Code → extension host → node.exe)

- **Startup app inventory:** Registry Run keys + Task Scheduler + Startup folder

Static hardware profile (collected once, refreshed on demand)

- CPU model, core count, base/boost clock
 - RAM total and speed
 - Storage devices (type: SSD/HDD/NVMe, capacity, interface)
 - GPU model
 - OS version and build number
 - BIOS/firmware version
-

Storage & Data Model

Design principles

- **SQLite** for ingestion (append-heavy writes, low overhead)
- **DuckDB** for analytical queries (correlations, aggregations, reports)
- **Retention policy:** raw metrics 30 days, hourly rollups 1 year, events indefinitely (configurable)
- **Compression:** downsample 1s → 1min → 1hr aggregates via background job

Core tables

```
-- Time-series metrics
metrics_raw (
  timestamp      INTEGER NOT NULL,    -- Unix ms
  metric_name     TEXT NOT NULL,      -- e.g. 'cpu.utilization.total'
  value          REAL NOT NULL,
  tags           TEXT                 -- JSON: {"core": 0, "disk": "C:"}
)

-- Discrete events
events (
  id             INTEGER PRIMARY KEY,
  timestamp      INTEGER NOT NULL,
  event_type     TEXT NOT NULL,      -- 'process.start', 'log.error',
'update.installed'
  source         TEXT,               -- 'Code.exe', 'System', etc.
  severity       TEXT,               -- 'info', 'warning', 'error'
  payload        TEXT                 -- JSON with event-specific fields
)

-- Rolling baselines (computed daily)
baselines (
  metric_name    TEXT NOT NULL,
  period        TEXT NOT NULL,      -- '1h', '1d', '7d'
  p50           REAL,
  p95           REAL,
  p99           REAL,
  stddev        REAL,
```

```

sample_count INTEGER,
updated_at   INTEGER
)

-- Pre/post change snapshots
snapshots (
  id          INTEGER PRIMARY KEY,
  label       TEXT,                -- 'before_kb5034441', 'after_nvidia_551.23'
  created_at  INTEGER,
  metrics     TEXT,                -- JSON blob of key metrics at snapshot time
  hardware    TEXT                 -- JSON hardware profile
)

-- Diagnosis audit trail
diagnoses (
  id          INTEGER PRIMARY KEY,
  query       TEXT,                -- user's original question
  timestamp   INTEGER,
  causes     TEXT,                -- JSON ranked causes with evidence
  explanation TEXT,                -- LLM-generated plain English
  confidence  REAL
)

-- Static system info
hardware_profile (
  key         TEXT PRIMARY KEY,
  value       TEXT,
  collected_at INTEGER
)

```

Analysis Engine

The analysis engine is the core intellectual property of this project. It operates in four stages:

1. Baseline learning

- Compute rolling statistics (median, p95, stddev) for each metric over 1h, 1d, and 7d windows
- Establish per-process baselines (e.g., "Code.exe normally uses 800 MB RAM at startup")
- Detect **concept drift** after OS/driver updates by comparing post-snapshot metrics to pre-snapshot baselines

2. Anomaly detection

Layered approach — simple methods first, ML later:

Method	Use case	Implementation
Threshold rules	Disk < 5% free, RAM available < 500 MB	Hard-coded guardrails
Z-score / IQR	Metric deviates from 7-day baseline	scipy.stats or manual
Change-point detection	Performance shift after update	CUSUM or ruptures library

Method	Use case	Implementation
Co-occurrence patterns	Multiple metrics spike together	Rule engine (Phase 1), association rules (Phase 3)
Isolation Forest	Multivariate anomaly scoring	Phase 3, once enough local data exists

3. Correlation engine

When a user asks about an app or an anomaly fires:

1. **Resolve entity**: "VS Code" → `Code.exe` (fuzzy match against running/recent processes)
2. **Define time window**: last launch, last 30 minutes, or user-specified
3. **Join events + metrics**: align process lifecycle with concurrent system state
4. **Build evidence graph**: nodes = metrics/events, edges = temporal proximity

4. Hypothesis scoring

Pre-defined hypothesis templates, each with required evidence and a scoring function:

Hypothesis	Evidence signals	Score weight
RAM shortage	Available RAM < threshold, page faults elevated, swap active	High
Thermal throttling	CPU temp high, frequency below base clock, power limit flag	High
Disk bottleneck	Queue depth > 5, disk latency > 20ms, high I/O during event	High
Disk space exhaustion	Volume > 90% full, temp file errors in event log	Medium
Heavy extension/subprocess	Child process RAM/CPU >> parent baseline	Medium
Startup app contention	Many apps launching concurrently, high CPU at boot	Medium
Background update/scan	Windows Update, Defender, or indexer active during event	Medium
Driver issue	Recent driver change + new errors in event log	Low (needs snapshot)
Network saturation	High bandwidth usage during perceived lag	Low

Final output: ranked list of { hypothesis, confidence, evidence[], suggested_actions[] }.

AI Agent Layer

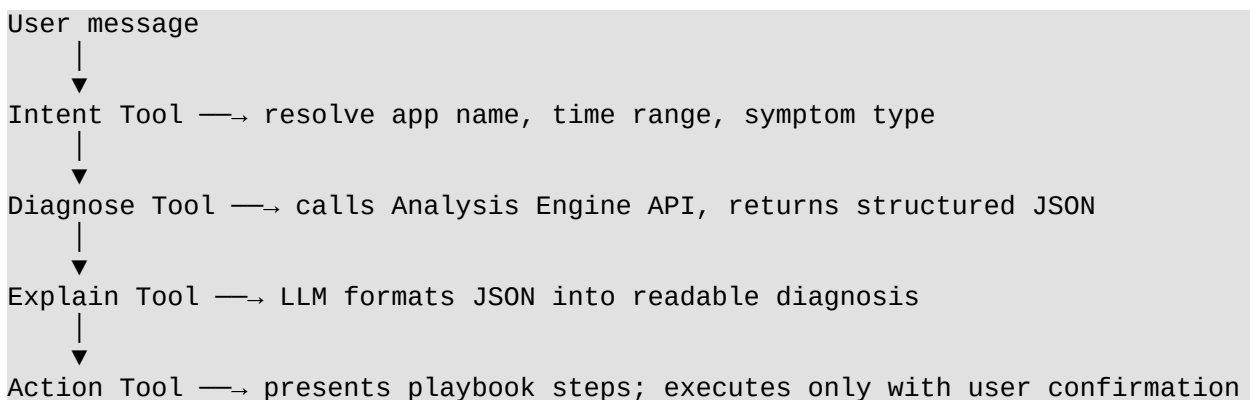
What the LLM does

- Parse natural language intent ("VS Code", "when gaming", "since yesterday's update")
- Generate plain-English explanation from structured diagnosis JSON
- Format remediation steps from a **fixed playbook** (never invent new fixes)
- Handle follow-up questions with conversation context

What the LLM does NOT do

- Access raw telemetry directly
- Invent metrics or root causes not present in the diagnosis payload
- Run system commands without explicit user approval
- Send any data off-machine (unless user opts into cloud LLM with informed consent)

Agent architecture



Structured diagnosis payload (LLM input):

```
{
  "query": "Why does VS Code lag on startup?",
  "resolved_entity": "Code.exe",
  "time_window": { "start": "2026-06-29T14:02:00Z", "end": "2026-06-29T14:03:30Z" },
  "severity": "high",
  "causes": [
    {
      "hypothesis": "extension_host_memory_pressure",
      "confidence": 0.87,
      "evidence": [
        { "metric": "process.memory.node.exe", "value": 1843, "unit": "MB", "baseline_p95": 620 },
        { "metric": "memory.available", "value": 412, "unit": "MB", "baseline_p50": 3200 }
      ],
      "actions": ["disable_extension:ms-python.python", "increase_available_ram"]
    },
    {
      "hypothesis": "disk_space_exhaustion",
```

```
    "confidence": 0.72,
    "evidence": [
      { "metric": "disk.free_percent.C", "value": 4.1, "unit": "%",
"threshold": 10 }
    ],
    "actions": ["free_disk_space", "run_disk_cleanup"]
  }
]
```

User Experience

MVP screens

1. System Tray Icon

- Green / yellow / red health indicator
- Quick stats: CPU, RAM, disk
- "Ask a question" shortcut

1. Dashboard

- Real-time metric charts (last 1h, 24h, 7d)
- Recent events timeline (process launches, errors, updates)
- Active anomalies list

1. Ask Interface

- Chat-style input for natural language questions
- Diagnosis card with ranked causes, evidence charts, and action buttons
- "Show me the data" expands raw metrics for transparency

1. Performance Report (24h)

- Auto-generated summary: health score, top resource consumers, anomalies detected
- Exportable as PDF or shareable snapshot

1. Snapshot Manager (Phase 2)

- Manual or automatic snapshots before/after updates
- Side-by-side comparison view

Tech Stack

Layer	Technology	Rationale
Collector service	Rust	Memory-safe, fast, excellent Windows API bindings

Layer	Technology	Rationale
		(windows - rs)
Analysis engine	Python 3.12+	pandas, scipy, rich ML/stats ecosystem
Analytical queries	DuckDB	Fast columnar queries over local data, SQL interface
Ingestion store	SQLite (via rusqlite)	Zero-config, embedded, proven for time-series with indexes
API server	FastAPI	Async, auto-docs, easy Python integration
Agent orchestration	LangGraph	Structured tool-calling workflow with state
LLM	Local (Ollama) or OpenAI API (user choice)	Local-first default; cloud opt-in
Desktop UI	Tauri 2.0 (Rust + Web frontend)	Lightweight vs Electron, native system tray
Frontend	React + TypeScript + Recharts	Dashboard charts and chat interface
Packaging	WiX / NSIS installer	Windows service + tray app bundle
CI/CD	GitHub Actions	Build, test, lint across Rust + Python + TS

Project Structure

```
pc-performance-copilot/
├── README.md
├── docs/
│   ├── architecture.md
│   ├── data-model.md
│   ├── hypothesis-catalog.md
│   └── demo-script.md
├── collector/                                # Rust – OS telemetry service
│   ├── src/
│   │   ├── main.rs
│   │   ├── metrics/                # CPU, RAM, disk, network collectors
│   │   ├── events/                # Process monitor, event log watcher
│   │   ├── etw/                   # ETW trace sessions (on-demand)
│   │   ├── hardware/              # Static profile collection
│   │   └── writer/                 # SQLite ingestion
│   └── Cargo.toml
├── analyzer/                                # Python – detection & correlation
│   ├── src/
│   │   ├── baselines.py
│   │   ├── anomalies.py
│   │   ├── correlation.py
│   │   ├── hypotheses.py
│   │   └── schemas.py
└── tests/
```

```

├── pyproject.toml
├── api/                                # FastAPI – REST endpoints
│   ├── src/
│   │   ├── main.py
│   │   ├── routes/
│   │   └── models/
│   └── pyproject.toml
├── agent/                              # LangGraph – NL explanation layer
│   ├── src/
│   │   ├── graph.py
│   │   ├── tools.py
│   │   └── prompts.py
│   └── pyproject.toml
├── app/                                # Tauri – desktop UI
│   ├── src/                            # React frontend
│   ├── src-tauri/                      # Tauri backend
│   └── package.json
├── playbook/                           # Remediation action definitions
│   └── actions.yaml
├── scripts/
│   ├── setup.ps1
│   ├── seed_demo_data.py              # Synthetic data for demo/testing
│   └── benchmark_scenarios/           # Reproducible lag scenarios

```

Development Plan

Phase 0 — Foundation (Week 1)

Goal: Repo scaffolding, CI, and a collector that writes one metric to disk.

- ☐ Initialize monorepo structure (collector, analyzer, api, agent, app)
- ☐ Set up GitHub Actions: Rust lint/test, Python lint/test, TS lint/build
- ☐ Implement SQLite schema and migration runner
- ☐ Collector: CPU + RAM polling every 5s → SQLite
- ☐ CLI tool to query latest metrics (`copilot-cli status`)

Deliverable: Background service that silently collects and stores CPU/RAM data.

Phase 1 — MVP Core (Weeks 2–4)

Goal: End-to-end diagnosis for one demo scenario (VS Code startup lag).

Week 2: Full collection + events

- ☐ Disk I/O, disk space, network collectors
- ☐ Process monitor (start/stop/top-N by CPU/RAM)
- ☐ Windows Event Log watcher (errors and warnings)
- ☐ Startup app inventory
- ☐ Hardware profile collection (one-shot)

- ☐ FastAPI endpoints: GET /metrics, GET /events, GET /health

Week 3: Analysis engine

- ☐ Baseline computation (rolling 7-day p50/p95)
- ☐ Threshold-based anomaly detection (disk, RAM, CPU)
- ☐ Z-score anomaly detection against baselines
- ☐ Correlation engine: join process launch with concurrent metrics
- ☐ Hypothesis scorer with 5 core templates (RAM, disk, thermal, extension, startup)
- ☐ POST /diagnose endpoint returning structured JSON

Week 4: Agent + minimal UI

- ☐ LangGraph agent: intent → diagnose → explain workflow
- ☐ Playbook YAML with 10 remediation actions
- ☐ Tauri tray app with system health indicator
- ☐ Chat interface: ask a question → see diagnosis card
- ☐ Seed demo data script for reproducible demo without waiting 7 days

Deliverable: Ask "Why does VS Code lag?" → get evidence-backed diagnosis with suggested fixes.

Phase 2 — Product polish (Weeks 5–7)

Goal: Make it feel like a real product, not a hackathon project.

- ☐ Dashboard with real-time charts (CPU, RAM, disk, network — 1h/24h/7d)
- ☐ Events timeline view (process launches, errors, anomalies)
- ☐ 24-hour Performance Report (auto-generated, PDF export)
- ☐ Snapshot system: manual + auto-trigger on Windows Update / driver change
- ☐ Before/after snapshot comparison view
- ☐ ETW on-demand deep trace for complex diagnoses
- ☐ Installer (Windows service + tray app)
- ☐ Demo video (60 seconds)

Deliverable: Polished demo suitable for resume link and LinkedIn post.

Phase 3 — Differentiation (Weeks 8–10)

Goal: Features that separate this from basic monitoring tools.

- ☐ Cohort benchmarking: compare metrics against similar hardware profiles
 - Seed cohort data from public hardware specs + synthetic baselines
 - "Your VS Code startup is slower than 82% of similar configs (16GB RAM, SSD, i7)"
- ☐ Change-point detection after OS/driver updates
- ☐ Game FPS correlation (PresentMon integration for frame time + GPU metrics)
- ☐ Background download / Windows Update overlap detection
- ☐ Antivirus scan contention detection
- ☐ Isolation Forest multivariate anomaly scoring
- ☐ Optional anonymized telemetry upload for cohort database (opt-in)

Deliverable: Benchmarking and regression tracking features for deeper interview conversations.

Phase 4 — Cross-platform (Weeks 11+)

Goal: Expand platform support (lower priority, resume v1 as Windows-only).

- ☐ macOS collector (`psutil`, `powermetrics`, IOKit for thermals)
 - ☐ Linux collector (`/proc`, `/sys`, eBPF probes)
 - ☐ Abstract collector trait/interface shared across platforms
-

MVP Success Criteria

The MVP (Phase 1) is done when all of the following are true:

1. **Collector runs silently** as a Windows background service, storing metrics and events locally
 2. **At least 5 metric types** collected: CPU, RAM, disk I/O, disk space, process stats
 3. **Baseline computation** runs daily and produces p50/p95 for key metrics
 4. **Anomaly detection** fires on at least 3 scenario types: RAM pressure, disk full, high CPU
 5. **Diagnosis endpoint** returns ranked causes with evidence for a targeted app query
 6. **LLM explanation** is grounded — every sentence maps to a field in the diagnosis JSON
 7. **Demo scenario works end-to-end:** VS Code startup lag → question → diagnosis → suggested fix
 8. **No data leaves the machine** unless user explicitly opts into cloud LLM
-

Demo Scenarios

Reproducible scenarios for testing and demo video:

#	Scenario	How to trigger	Expected diagnosis
1	VS Code slow startup	Install heavy extension, open large workspace	Extension memory pressure + disk I/O
2	System-wide lag	Fill RAM with memory hog, then open any app	RAM shortage + swap thrashing
3	Disk bottleneck	Copy large files while launching apps	Disk queue depth spike
4	Post-update regression	Take snapshot, install Windows update, compare	Change-point detected, metric drift
5	Startup slowness	Enable 20+ startup apps, reboot	Startup app contention
6	Thermal throttle	Run CPU stress test, then launch app	CPU frequency below base clock

Each scenario has a script in `scripts/benchmark_scenarios/` that sets up conditions and validates expected diagnosis output.

Privacy & Security

- **Local-first by default** — all telemetry stored in `%LOCALAPPDATA%\PCCopilot\`
- **No cloud upload** unless user explicitly opts in (Phase 3 cohort feature)
- **LLM local by default** — Ollama with a small model; cloud API is opt-in with clear disclosure
- **No keystroke logging, no screen capture, no browser history**
- **Process names only** — no window titles or file paths in stored data (configurable)
- **Admin elevation** requested only for ETW traces and Event Log access; app degrades gracefully without it
- **Open source** — users can audit exactly what is collected
- **One-click purge** — delete all stored data from settings

Future Roadmap

- ☐ **Predictive alerts** — "Based on disk usage trend, C: will be full in 12 days"
- ☐ **Automated remediation** — one-click "fix" actions (disable startup app, clear temp files) with user confirmation

- ☐ **Multi-machine dashboard** — optional sync across user's devices (requires opt-in cloud)
 - ☐ **Integration with Task Manager / Resource Monitor** — deep-link to OS tools from diagnosis
 - ☐ **Plugin system** — community-contributed hypothesis templates and playbook actions
 - ☐ **Mobile companion** — push alerts when home PC has issues (stretch goal)
-

Skills Demonstrated

Skill domain	How this project demonstrates it
Operating systems	WMI, ETW, Performance Counters, Event Log, process lifecycle, memory management
Systems programming	Rust collector service, Windows API bindings, efficient polling
Data engineering	Time-series ingestion, retention policies, rollups, schema design, ETL pipeline
Data analysis	Baseline computation, anomaly detection, correlation, change-point detection
Search & recommendation	Cohort matching, hardware profile similarity, benchmark ranking
AI agents	LangGraph orchestration, tool-calling, grounded generation, playbook-driven actions
Full-stack development	Tauri desktop app, FastAPI backend, React dashboard
DevOps	CI/CD, installer packaging, service management

Getting Started

Status: Pre-development. Phase 0 not yet started.

Prerequisites

- Windows 10/11 (64-bit)
- Rust 1.77+ (rustup)
- Python 3.12+ (uv or pip)
- Node.js 20+ (pnpm)
- Visual Studio Build Tools (for Rust Windows bindings)

Setup (coming soon)

```
# Clone the repository
git clone https://github.com/<your-username>/pc-performance-copilot.git
cd pc-performance-copilot

# Run setup script (installs deps, creates DB, registers service)
.\scripts\setup.ps1

# Start the collector service
.\collector\target\release\copilot-collector.exe

# Start the API server
cd api && uvicorn src.main:app --reload

# Start the desktop app
cd app && pnpm tauri dev
```

Running the demo

```
# Seed synthetic baseline data (simulates 7 days of normal operation)
python scripts/seed_demo_data.py

# Trigger the VS Code lag scenario
.\scripts\benchmark_scenarios\vscode_lag.ps1

# Ask the copilot
curl -X POST http://localhost:8000/diagnose \
  -H "Content-Type: application/json" \
  -d '{"query": "Why does VS Code lag on startup?"}'
```

License

MIT License — see [LICENSE](#) for details.

Author

Radley Le — SWE / Data / AI Engineer

Building tools that sit at the intersection of systems software, data pipelines, and intelligent interfaces.